



Sockets em Jogos Multiplayer

Aplicação no projeto Pici Hunt



O que é um Socket?

Definição

Socket é um mecanismo de comunicação que permite a troca de dados entre programas através de uma rede, utilizando IP, porta e protocolos como TCP ou UDP.

Analogia

Pense em um Socket como um **telefone** que conecta duas fazendas, permitindo que elas conversem usando endereços (IP) e números de ramal (Porta).



Cliente



Socket



Rede



Socket



Servidor

IP e Porta

CONCEITO	FUNÇÃO
IP	Endereço da Fazenda (identificador numérico)
Porta	Entrada Específica (número lógico)
Socket	Canal de Comunicação

Exemplo no Pici H

192.168.0.15:55000

📍 **IP:**192.168.0.15 (Sua Fazenda Host)

🚪 **Porta:**55000 (A Entrada do Jogo)

Modelo Cliente-Servidor



Servidor (Fazenda Principal)

A fazenda principal que espera e recebe os jogadores (clientes) que querem participar do jogo.

```
net.startServer();
```

Cliente (Jogador)

O jogador que inicia a conexão para entrar na fazenda principal e jogar.

```
net.connect(ip);
```

TCP vs UDP

CARACTERÍSTICA	TCP (ESTRADA DE PEDRA)	UDP (ATALHO DE TERRA)
Confiabilidade	Confiável (Carteiro Garante)	Pode perder dados (Mensagem Gritada)
Entrega	Garante entrega	Não garante
Ordem	Mantém ordem	Não mantém ordem
Velocidade	Mais lento	Menor latência (rápido)

O Projeto Usa TCP



`socket(AF_INET, SOCK_STREAM, 0)` = TCP

Por que TCP? (A Carta Registrada)

*"Como o jogo é por turnos, perder mensagens seria problemático.
TCP garante entrega correta dos ataques e resultados."*

Fluxo da Conexão

Host (Servidor)

 `socket()` Cria o canal

 `bind()` Liga a porta

 `listen()` Espera clientes

 `accept()` Aceita conexão

Cliente

`socket()` Cria o canal

 `connect()` Conecta

Comunicação

 `send()`

 `recv()`

Estrutura dos Pacotes

As Mensagens do Jogo

No Pici Hunt, o jogo envia **pacotes binários**, que são como pequenas cartas com informações importantes:



Ataque, Resultado, Game Over!

Eles são organizados como uma

```
struct Packet .
```

TIPO DE PACOTE	FUNÇÃO
 ATTACK	Ação de ataque do jogador
 RESULT	Resposta do ataque ou evento
 READY	Jogador pronto para a próxima rodada
 GAMEOVER	Indica o fim da partida

Comunicação no Jogo (Fluxo Real)



Jogador ataca



Envia mensagem com `send()`



Outro jogador recebe com

`recv()`



Processa ataque



Envia resultado

Sockets Não-Bloqueantes

`O_NONBLOCK`

Sem Non-blocking

O jogo travaria, esperando a rede.
Imagine seu personagem parado, sem
fazer nada!



Com Non-blocking

- 🌱 Render continua
- 🏃 Jogo continua
- 🔄 Rede é verificada continuamente

Conclusão

Construindo a Fazenda Multiplayer do Pici Hunt

- **Sockets:** A base para o nosso mundo multiplayer.
- **TCP:** Garantiu que cada ação e resultado chegasse em segurança.
- **Cliente-Servidor:** Uma arquitetura robusta para comunicação fluida.
- **Pacotes Binários:** Mensagens otimizadas para o jogo.
- Resultado: Uma experiência de jogo **fluida, confiável e divertida** para todos os fazendeiros!